

Python Basics for Machine Learning

Complete guide to Python fundamentals needed for machine learning and data science —**AI programming with Python**: syntax, functional patterns, OOP, files, and exceptions.

Table of Contents

- [Comments](#)
 - [Variables and Data Types](#)
 - [In-Depth Strings](#)
 - [Print Statements](#)
 - [Type Conversion and Truthy/Falsy Values](#)
 - [Input Statements](#)
 - [Control Flow](#)
 - [Functions](#)
 - [Data Structures](#)
 - [File I/O](#)
 - [Tell, seek, and read position](#)
 - [Error Handling](#)
 - [Object-Oriented Programming](#)
 - [Practice Exercises](#)
 - [Capstone: Movie script generator](#)
-

Comments

Comments are notes in your code that Python ignores. They help explain what your code does.

Single-Line Comments

Use `#` for single-line comments:

```
# This is a comment
name = "Alice" # This is also a comment

# Comments can be on their own line
age = 25
```

Multi-Line Comments

Use triple quotes `"""` or `'''` for multi-line comments:

```
"""
This is a multi-line comment.
It can span multiple lines.
Useful for documentation.
"""

'''
This is also a multi-line comment.
Both triple single and double quotes work.
'''
```

Best Practices

- Write clear, concise comments
- Explain "why" not "what" (code should be self-explanatory)
- Update comments when code changes
- Don't over-comment obvious code

Variables and Data Types

Variables

Variables store data values. In Python, you don't need to declare variable types.

```
# Assigning values
name = "Alice"
age = 25
height = 5.6
```

```
is_student = True

print(name)      # Output: Alice
print(age)       # Output: 25
print(height)    # Output: 5.6
print(is_student) # Output: True
```

Variable Naming Conventions

Python has several naming conventions:

1. Snake Case (Recommended for Python)

```
# Use lowercase with underscores
my_variable = 10
user_name = "Alice"
total_count = 100
```

2. Camel Case

```
# First word lowercase, subsequent words capitalized
myVariable = 10
userName = "Alice"
totalCount = 100
```

3. Pascal Case (Used for Classes)

```
# All words capitalized
MyVariable = 10
UserName = "Alice"
TotalCount = 100
```

Variable Naming Rules

1. Must start with a letter or underscore - Cannot start with a number

```
valid_name = "OK"
_valid = "OK"
# 2invalid = "Error!" # This will cause an error
```

2. Can contain letters, numbers, and underscores - No spaces or special characters (except `_`)

```
user_name = "OK"  
user_name_2 = "OK"  
# user name = "Error!" # Spaces not allowed  
# user-name = "Error!" # Hyphens not allowed
```

3. Case-sensitive - `name` and `Name` are different variables

```
name = "Alice"  
Name = "Bob"  
print(name) # Output: Alice  
print(Name) # Output: Bob
```

4. Cannot use Python keywords - Don't use words like `if`, `for`, `def`, `class`, etc.

```
# if = 10 # Error! 'if' is a keyword  
# def = "test" # Error! 'def' is a keyword
```

Common Python Keywords

```
# These are reserved words in Python  
# False, None, True, and, as, assert, async, await, break, class, continue, def,  
del, elif, else, except, finally, for, from, global, if, import, in, is, lambda,  
nonlocal, not, or, pass, raise, return, try, while, with, yield
```

```
# Assigning values  
name = "Alice"  
age = 25  
height = 5.6  
is_student = True  
  
print(name) # Output: Alice  
print(age) # Output: 25  
print(height) # Output: 5.6  
print(is_student) # Output: True
```

Data Types

1. Numbers

```
# Integers
x = 10
y = -5

# Floats (decimals)
pi = 3.14
temperature = -10.5

# Complex numbers
z = 3 + 4j

# Type checking
print(type(x)) # Output: <class 'int'>
print(type(pi)) # Output: <class 'float'>
```

2. Strings

```
# Single or double quotes
name1 = "Alice"
name2 = 'Bob'

# String operations
full_name = name1 + " " + name2 # Concatenation
print(full_name) # Output: Alice Bob

# String methods
text = "Hello World"
print(text.upper()) # Output: HELLO WORLD
print(text.lower()) # Output: hello world
print(text.replace("World", "Python")) # Output: Hello Python
print(len(text)) # Output: 11
```

3. Booleans

```
is_true = True
is_false = False

# Boolean operations
result = is_true and is_false # False
result = is_true or is_false # True
result = not is_true # False
```

4. Type Conversion

```
# Convert between types
x = "123"
y = int(x)      # Convert to integer: 123
z = float(x)    # Convert to float: 123.0
w = str(123)    # Convert to string: "123"
```

In-Depth Strings

String Indexing

Strings are sequences of characters. Each character has an index (position).

Positive Indexing (Left to Right):

```
text = "hello"
# Index:  0  1  2  3  4
# Value:  h  e  l  l  o

print(text[0]) # Output: h
print(text[1]) # Output: e
print(text[4]) # Output: o
```

Negative Indexing (Right to Left):

```
text = "hello"
# Index:  -5 -4 -3 -2 -1
# Value:   h  e  l  l  o

print(text[-1]) # Output: o (last character)
print(text[-2]) # Output: l (second to last)
print(text[-5]) # Output: h (first character)
```

String Slicing

Extract portions of strings using `[start:stop:step]`:

```
text = "yoo brother"

# Basic slicing
```

```
print(text[0:3])    # Output: yoo (indices 0, 1, 2)
print(text[4:11])   # Output: brother (indices 4 to 10)
print(text[:3])     # Output: yoo (from start to index 2)
print(text[4:])     # Output: brother (from index 4 to end)
print(text[:])      # Output: yoo brother (entire string)

# With step
print(text[0:11:2]) # Output: yo rte (every 2nd character)
print(text[::-2])   # Output: yo rte (every 2nd character from start to end)
print(text[::-1])   # Output: rehtorb ooy (reverse string)
```

String Immutability

Strings are **immutable** - they cannot be changed after creation:

```
text = "hello"
# text[0] = "H" # Error! Cannot modify string

# Instead, create a new string
text = "H" + text[1:] # Creates new string: "Hello"
print(text) # Output: Hello
```

Why Immutability Matters:

- Strings are safe to pass around (can't be accidentally modified)
- More memory efficient (can be shared between variables)
- Required for dictionary keys (keys must be immutable)

Following is a way to deal with it:

```
chars = list("hello")
chars[0] = 'H' # lists ARE mutable
result = ''.join(chars) # convert back to string once at the end
```

Print Statements

Basic Print

```
print("Hello, World!")
print(42)
print(3.14)
```

Print Multiple Items

```
name = "Alice"
age = 25
print("Name:", name, "Age:", age) # Output: Name: Alice Age: 25
```

String Concatenation

```
first_name = "Alice"
last_name = "Smith"
full_name = first_name + " " + last_name
print(full_name) # Output: Alice Smith

# Note: Can only concatenate strings with strings
# print("Age: " + 25) # Error! Need to convert to string
print("Age: " + str(25)) # Output: Age: 25
```

Formatted Strings (f-strings)

f-strings (formatted string literals) are the modern way to embed variables:

```
name = "Alice"
age = 25
city = "New York"

# f-string syntax
message = f"Hello, {name}! You are {age} years old and live in {city}."
print(message) # Output: Hello, Alice! You are 25 years old and live in New York.

# Can include expressions
print(f"Next year you'll be {age + 1}") # Output: Next year you'll be 26

# Formatting numbers
pi = 3.14159
print(f"Pi is approximately {pi:.2f}") # Output: Pi is approximately 3.14
```


Escape Sequences

Special characters in strings:

```
# Newline: \n
print("Line 1\nLine 2")
# Output:
# Line 1
# Line 2

# Tab: \t
print("Name:\tAlice")
# Output: Name:    Alice

# Backspace: \b
print("Hello\bWorld") # Output: HellWorld (removes 'o')

# Backslash: \\
print("Path: C:\\Users\\Alice") # Output: Path: C:\Users\Alice

# Single quote: \'
print('It\'s a beautiful day') # Output: It's a beautiful day

# Double quote: \"
print("She said \"Hello\"") # Output: She said "Hello"
```

Raw Strings

Raw strings ignore escape sequences (useful for file paths, regex):

```
# Regular string
path1 = "C:\\Users\\Alice\\Documents" # Need double backslashes
print(path1) # Output: C:\Users\Alice\Documents

# Raw string (prefix with 'r')
path2 = r"C:\Users\Alice\Documents" # No need to escape
print(path2) # Output: C:\Users\Alice\Documents

# Useful for regex patterns
import re
pattern = r"\d+" # Matches one or more digits
```

Type Conversion and Truthy/Falsy

Values

Type Conversion Functions

```
# Convert to integer
x = int("123")      # 123
x = int(3.14)       # 3 (truncates decimal)
# x = int("abc")    # Error! Cannot convert

# Convert to float
y = float("3.14")   # 3.14
y = float(5)        # 5.0

# Convert to string
z = str(123)         # "123"
z = str(3.14)        # "3.14"
z = str(True)        # "True"

# Convert to boolean
w = bool(1)          # True
w = bool(0)          # False
w = bool("hello")    # True
w = bool("")         # False
```

Truthy and Falsy Values

In Python, values are evaluated as **True** or **False** in boolean contexts.

Falsy Values (evaluate to **False**):

```
# All of these are falsy
bool(False)        # False
bool(None)         # False
bool(0)            # False
bool(0.0)          # False
bool(0j)           # False (complex zero)
bool("")           # False (empty string)
bool([])           # False (empty list)
bool({})           # False (empty dictionary)
bool(())           # False (empty tuple)
bool(set())        # False (empty set)
```

Truthy Values (evaluate to **True**):

```
# Everything else is truthy
bool(True)      # True
bool(1)         # True
bool(-1)        # True
bool(3.14)      # True
bool("hello")   # True
bool([1, 2, 3]) # True
bool({"a": 1})  # True
```

Practical Examples:

```
# Check if list is not empty
my_list = []
if my_list: # Falsy, so this won't execute
    print("List has items")
else:
    print("List is empty") # This executes

# Check if string is not empty
name = input("Enter name: ")
if name: # Truthy if name has characters
    print(f"Hello, {name}!")
else:
    print("No name provided")

# Default value pattern
value = None
result = value or "default" # Uses "default" if value is falsy
print(result) # Output: default
```

Input Statements

The `input()` function gets user input from the keyboard.

Basic Input

```
# input() always returns a string
name = input("What's your name? ")
print(f"Hello, {name}!")
```

Type Conversion with Input

Since `input()` returns a string, convert it for numeric operations:

```
# Get age as integer
age = int(input("Tell your age: "))
print(f"Next year you'll be {age + 1}")

# Get price as float
price = float(input("Enter price: "))
print(f"Price with tax: ${price * 1.1:.2f}")

# Get boolean (yes/no)
response = input("Do you like Python? (yes/no): ")
likes_python = response.lower() == "yes"
print(f"Likes Python: {likes_python}")
```

Handling Input Errors

```
# Safe input with error handling
while True:
    try:
        age = int(input("Enter your age: "))
        if age > 0:
            break
        else:
            print("Age must be positive!")
    except ValueError:
        print("Please enter a valid number!")

print(f"You entered: {age}")
```

Multiple Inputs

```
# Get multiple values
name = input("Enter name: ")
age = int(input("Enter age: "))
city = input("Enter city: ")

print(f"{name} is {age} years old and lives in {city}")

# Get multiple values on one line (comma-separated)
data = input("Enter name, age, city (comma-separated): ")
name, age, city = data.split(",")
age = int(age.strip())
city = city.strip()
print(f"{name} is {age} years old and lives in {city}")
```

Control Flow

If/Else Statements

```
age = 18

if age >= 18:
    print("You are an adult")
elif age >= 13:
    print("You are a teenager")
else:
    print("You are a child")

# Output: You are an adult
```

Comparison Operators:

```
x = 10
y = 5

print(x > y)    # True
print(x < y)    # False
print(x == y)   # False (equality)
print(x != y)   # True (not equal)
print(x >= y)   # True
print(x <= y)   # False
```

Loops

For Loop:

```
# Iterate over a list
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

# Output:
# apple
# banana
# cherry

# Using range
```

```
for i in range(5):
    print(i)

# Output: 0, 1, 2, 3, 4

# With index
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")

# Output:
# 0: apple
# 1: banana
# 2: cherry
```

While Loop:

```
count = 0
while count < 5:
    print(count)
    count += 1

# Output: 0, 1, 2, 3, 4
```

Loop Control:

```
# Break: exit loop
for i in range(10):
    if i == 5:
        break
    print(i)
# Output: 0, 1, 2, 3, 4

# Continue: skip iteration
for i in range(5):
    if i == 2:
        continue
    print(i)
# Output: 0, 1, 3, 4
```

Functions

Defining Functions

```
def greet(name):  
    """This function greets a person"""  
    return f"Hello, {name}!"  
  
message = greet("Alice")  
print(message) # Output: Hello, Alice!
```

Function Parameters

```
# Default parameters  
def power(base, exponent=2):  
    return base ** exponent  
  
print(power(3))      # Output: 9 (3^2)  
print(power(3, 3))   # Output: 27 (3^3)  
  
# Keyword arguments  
def introduce(name, age, city):  
    return f"{name} is {age} years old and lives in {city}"  
  
print(introduce(age=25, city="New York", name="Alice"))  
# Output: Alice is 25 years old and lives in New York
```

Lambda Functions

```
# Anonymous functions  
square = lambda x: x ** 2  
print(square(5)) # Output: 25  
  
# Common use: with map, filter  
numbers = [1, 2, 3, 4, 5]  
squared = list(map(lambda x: x**2, numbers))  
print(squared) # Output: [1, 4, 9, 16, 25]  
  
evens = list(filter(lambda x: x % 2 == 0, numbers))  
print(evens) # Output: [2, 4]  
  
# reduce: combine iterable into one value (from functools)  
from functools import reduce  
  
product = reduce(lambda acc, x: acc * x, numbers, 1)  
print(product) # Output: 120 (1*2*3*4*5)
```

Data Structures

Lists

```
# Creating lists
fruits = ["apple", "banana", "cherry"]
numbers = [1, 2, 3, 4, 5]

# Accessing elements
print(fruits[0])      # Output: apple (first element)
print(fruits[-1])     # Output: cherry (last element)

# Slicing
print(numbers[1:3])   # Output: [2, 3] (index 1 to 2)
print(numbers[:3])    # Output: [1, 2, 3] (first 3)
print(numbers[2:])    # Output: [3, 4, 5] (from index 2)

# Modifying lists
fruits.append("orange") # Add to end
fruits.insert(1, "grape") # Insert at index
## fruits=['apple', 'grape', 'banana', 'cherry', 'orange']
fruits.remove("banana") # Remove element
## fruits=['apple', 'grape', 'cherry', 'orange']
fruits.pop()           # Remove last element
## fruits=['apple', 'grape', 'cherry']

# List comprehension
squares = [x**2 for x in range(5)]
print(squares) # Output: [0, 1, 4, 9, 16]

# With condition
evens = [x for x in range(10) if x % 2 == 0]
print(evens) # Output: [0, 2, 4, 6, 8]
```

Dictionaries

```
# Creating dictionaries
student = {
    "name": "Alice",
    "age": 20,
    "grades": [85, 90, 88]
}

# Accessing values
print(student["name"]) # Output: Alice
print(student.get("age")) # Output: 20
print(student.get("city", "N/A")) # Output: N/A (default if key doesn't exist)
```



```
# Modifying dictionaries
student["city"] = "New York"      # Add/update
student["age"] = 21                # Update
del student["grades"]             # Delete key

# Iterating
for key, value in student.items():
    print(f"{key}: {value}")

# Dictionary comprehension
squares_dict = {x: x**2 for x in range(5)}
print(squares_dict)  # Output: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Tuples

```
# Tuples are immutable (cannot be changed)
coordinates = (10, 20)
point = (x, y) = (5, 3)

# Accessing
print(coordinates[0])  # Output: 10

# Unpacking
x, y = coordinates
print(x, y)  # Output: 10 20

# Use cases: when you need immutable data
colors = ("red", "green", "blue")
```

Sets

```
# Sets store unique elements
unique_numbers = {1, 2, 3, 3, 4}
print(unique_numbers)  # Output: {1, 2, 3, 4}

# Set operations
set1 = {1, 2, 3}
set2 = {3, 4, 5}

print(set1.union(set2))      # Output: {1, 2, 3, 4, 5}
print(set1.intersection(set2)) # Output: {3}
print(set1.difference(set2))  # Output: {1, 2}
```

File I/O

Reading Files

```
# Read entire file
with open("data.txt", "r") as file:
    content = file.read()
    print(content)

# Read line by line
with open("data.txt", "r") as file:
    for line in file:
        print(line.strip()) # strip() removes newline

# Read all lines into list
with open("data.txt", "r") as file:
    lines = file.readlines()
```

Writing Files

```
# Write to file
with open("output.txt", "w") as file:
    file.write("Hello, World!\n")
    file.write("This is a new line")

# Append to file
with open("output.txt", "a") as file:
    file.write("\nAppended text")
```

CSV Files

```
import csv

# Reading CSV
with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)

# Writing CSV
with open("output.csv", "w", newline="") as file:
    writer = csv.writer(file)
```

```
writer.writerow(["Name", "Age", "City"])
writer.writerow(["Alice", 25, "New York"])
```

Tell, seek, and read position

Text files are read sequentially. `tell()` returns the current byte offset in the file; `seek(offset, whence)` moves that position (`whence`: 0 start, 1 current, 2 end).

```
with open("data.txt", "w+") as f:
    f.write("Hello ML")
    f.seek(0)          # back to start for reading
    print(f.read())    # Hello ML
    print(f.tell())    # position after read (end of file)

with open("data.txt", "r") as f:
    chunk = f.read(5)   # first 5 characters
    print(f.tell())    # 5
    f.seek(0)          # jump to beginning again
    print(f.read())    # full file
```

Use `tell` / `seek` when you re-read parts of a file, build random-access parsers, or implement **practice problems based on file handling** (e.g., jump to a scene marker in a script file).

Error Handling

Try/Except

```
# Basic error handling
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")

# Multiple exceptions
try:
    value = int("not a number")
    result = 10 / value
except ValueError:
    print("Invalid number format")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

```

except Exception as e:
    print(f"An error occurred: {e}")

# Else: runs only if no exception was raised in try
try:
    value = int("42")
except ValueError:
    print("Not an integer")
else:
    print(f"Parsed OK: {value}")

# Finally: always runs (cleanup); use with open() when possible
file = None
try:
    file = open("data.txt", "r")
    content = file.read()
except FileNotFoundError:
    print("File not found")
finally:
    if file is not None:
        file.close()

```

Raising Exceptions

```

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b

try:
    result = divide(10, 0)
except ValueError as e:
    print(e) # Output: Cannot divide by zero

```

Object-Oriented Programming

Classes and Objects

```

class Dog:
    # Class attribute
    species = "Canis familiaris"

    # Constructor
    def __init__(self, name, age):
        self.name = name # Instance attribute

```

```

        self.age = age

# Method
def bark(self):
    return f"{self.name} says Woof!"

def get_info(self):
    return f"{self.name} is {self.age} years old"

# Creating objects
dog1 = Dog("Buddy", 3)
dog2 = Dog("Max", 5)

print(dog1.bark())      # Output: Buddy says Woof!
print(dog2.get_info())  # Output: Max is 5 years old

```

Inheritance

```

class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound"

class Cat(Animal):
    def speak(self): # Override parent method
        return f"{self.name} says Meow"

class Dog(Animal):
    def speak(self): # Override parent method
        return f"{self.name} says Woof"

cat = Cat("Whiskers")
dog = Dog("Buddy")

print(cat.speak()) # Output: Whiskers says Meow
print(dog.speak()) # Output: Buddy says Woof

```

Polymorphism

The same interface (e.g., method name `speak`) behaves differently per type—shown above where `Cat` and `Dog` override `speak`. You can write functions that accept any `Animal` without knowing the concrete class.

Encapsulation and abstraction

- **Encapsulation:** Bundle data and behavior in a class; hide internal state with conventions (single leading `_balance`) or descriptors; expose only methods you want callers to use.
- **Abstraction:** Model real-world ideas with simple interfaces (`deposit`, `withdraw`) while hiding implementation details.

```
class Counter:
    def __init__(self):
        self._count = 0 # "protected" by convention

    def increment(self):
        self._count += 1

    def value(self):
        return self._count
```

Practice Exercises

Exercise 1: Basic Operations

Task: Write a function that takes two numbers and returns their sum, difference, product, and quotient.

Solution:

```
def calculate(a, b):
    return {
        "sum": a + b,
        "difference": a - b,
        "product": a * b,
        "quotient": a / b if b != 0 else "Cannot divide by zero"
    }

result = calculate(10, 5)
print(result)
# Output: {'sum': 15, 'difference': 5, 'product': 50, 'quotient': 2.0}
```

Exercise 2: List Manipulation

Task: Write a function that takes a list of numbers and returns a new list with only even numbers squared.

Solution:

```
def even_squares(numbers):  
    return [x**2 for x in numbers if x % 2 == 0]  
  
result = even_squares([1, 2, 3, 4, 5, 6])  
print(result)  # Output: [4, 16, 36]
```

Exercise 3: Dictionary Operations

Task: Create a function that counts word frequency in a sentence.

Solution:

```
def word_count(sentence):  
    words = sentence.lower().split()  
    frequency = {}  
    for word in words:  
        frequency[word] = frequency.get(word, 0) + 1  
    return frequency  
  
result = word_count("the quick brown fox jumps over the lazy dog")  
print(result)  
# Output: {'the': 2, 'quick': 1, 'brown': 1, 'fox': 1, 'jumps': 1, 'over': 1,  
           'lazy': 1, 'dog': 1}
```

Exercise 4: File Processing

Task: Write a program that reads a file and counts the number of lines and words.

Solution:

```
def count_file_stats(filename):  
    try:  
        with open(filename, "r") as file:  
            lines = file.readlines()  
            line_count = len(lines)  
            word_count = sum(len(line.split()) for line in lines)  
            return {"lines": line_count, "words": word_count}  
    except FileNotFoundError:  
        return "File not found"
```

```
stats = count_file_stats("data.txt")
print(stats)
```

Exercise 5: Class Implementation

Task: Create a `BankAccount` class with deposit, withdraw, and balance methods.

Solution:

```
class BankAccount:
    def __init__(self, account_number, initial_balance=0):
        self.account_number = account_number
        self.balance = initial_balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            return f"Deposited ${amount}. New balance: ${self.balance}"
        return "Invalid deposit amount"

    def withdraw(self, amount):
        if 0 < amount <= self.balance:
            self.balance -= amount
            return f"Withdrew ${amount}. New balance: ${self.balance}"
        return "Insufficient funds or invalid amount"

    def get_balance(self):
        return f"Account {self.account_number} balance: ${self.balance}"

account = BankAccount("12345", 1000)
print(account.deposit(500))    # Output: Deposited $500. New balance: $1500
print(account.withdraw(200))  # Output: Withdrew $200. New balance: $1300
print(account.get_balance())  # Output: Account 12345 balance: $1300
```

Capstone: Movie script generator

Combine variables, control flow, lists/dictionaries, file I/O (including optional `seek/tell` for acts or scenes), and optionally classes (e.g., `Scene`, `Character`) to build a **movie script generator** that:

1. Reads templates or character bios from text/CSV files
2. Fills dialogue or stage directions from structured data or random pools
3. Writes a formatted script to a new `.txt` or `.fountain` file

This reinforces **file handling** and **practice problems based on file handling** before you move on to NumPy-heavy work.

Runnable example (CSV in → script out)

This example uses a **temporary folder** so you can run it anywhere. It writes a small **characters.csv**, reads it back, picks random lines, and writes **screenplay.txt**. It also shows **tell / seek** when re-reading the start of the CSV after the first pass.

```
import csv
import random
import tempfile
from pathlib import Path

def write_sample_cast_csv(path: Path) -> None:
    """Create a tiny cast file (what you might edit by hand)."""
    rows = [
        {"name": "Dr. Vega", "role": "scientist", "trait": "sarcastic"},
        {"name": "Unit 7", "role": "robot", "trait": "literal-minded"},
        {"name": "Mara", "role": "pilot", "trait": "reckless"},
    ]
    with path.open("w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=["name", "role", "trait"])
        writer.writeheader()
        writer.writerows(rows)

def load_cast(path: Path) -> list[dict]:
    with path.open("r", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        return list(reader)

def peek_file_start(path: Path, n_bytes: int = 80) -> bytes:
    """Demonstrate tell/seek: read a slice from the beginning without loading whole file."""
    with path.open("rb") as f:
        snippet = f.read(n_bytes)
        pos = f.tell()
        f.seek(0)
        again = f.read(n_bytes)
    assert snippet == again
    return snippet

def random_line(pool: list[str]) -> str:
    return random.choice(pool)

def build_script(cast: list[dict], out_path: Path) -> None:
    establishing = [
```

```

"INT. RESEARCH LAB - NIGHT",
"INT. STARSHIP BRIDGE - DAWN",
"EXT. CRATER RIM - DAY",
]

beats = [
    "A console blinks red. Nobody wants to be the first to speak.",
    "The ship shudders. Something massive passes the viewport.",
    "Silence, except for the hum of cooling fans.",
]

lines_out: list[str] = []
lines_out.append("TITLE: AUTOGENERATED SHORT")
lines_out.append("")
lines_out.append(random_line(establishing))
lines_out.append("")
lines_out.append(random_line(beats))
lines_out.append("")

for person in cast:
    name = person["name"]
    trait = person["trait"]
    dialogue = random_line(
        [
            f"We are not paid enough for this, {trait} or not.",
            "Run the diagnostic again. I do not trust that reading.",
            "If we survive, I am never flying coach again.",
        ]
    )
    lines_out.append(f"{name.upper()}")
    lines_out.append(f"    ({trait})")
    lines_out.append(f"    {dialogue}")
    lines_out.append("")

text = "\n".join(lines_out)
try:
    out_path.write_text(text, encoding="utf-8")
except OSError as e:
    print(f"Could not write script: {e}")
else:
    print(f"Wrote {out_path} ({len(text)} characters)")
finally:
    # Placeholder for cleanup (e.g., close remote handles); with
    Path.write_text nothing to close
    pass

def main() -> None:
    work = Path(tempfile.mkdtemp(prefix="movie_script_demo_"))
    csv_path = work / "characters.csv"
    script_path = work / "screenplay.txt"

    write_sample_cast_csv(csv_path)
    raw_head = peek_file_start(csv_path)
    print(f"First bytes of CSV (tell/seek demo):", raw_head[:40], "...")

    cast = load_cast(csv_path)
    random.seed() # omit for reproducible demos: set random.seed(0)
    build_script(cast, script_path)

```

```
if __name__ == "__main__":  
    main()
```

Try extending it: add a `Scene` class with `title` and `lines` list, read act markers from a second file and use `seek` to jump between sections, or swap the CSV for JSON.

Time Complexity and Algorithm Efficiency

Understanding time complexity is crucial for writing efficient code, especially when working with large datasets in data science and machine learning.

What is Time Complexity?

Time complexity describes how the runtime of an algorithm grows as the input size increases. We use Big O notation to express this.

Big O Notation

Big O notation describes the worst-case scenario for how long an algorithm takes.

Common Time Complexities:

1. $O(1)$ - Constant Time

- Accessing an element in a list by index
- Operations that take the same time regardless of input size

```
# O(1) - Constant time  
def get_first_element(lst):  
    return lst[0] # Always takes same time  
  
# Dictionary lookup is O(1) on average  
my_dict = {'a': 1, 'b': 2, 'c': 3}  
value = my_dict['a'] # O(1)
```

2. $O(\log n)$ - Logarithmic Time

- Binary search

- Operations that divide the problem in half each time

```
# O(log n) - Binary search
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

3. $O(n)$ - Linear Time

- Iterating through a list
- Operations that process each element once

```
# O(n) - Linear time
def find_max(lst):
    max_val = lst[0]
    for num in lst: # Process each element once
        if num > max_val:
            max_val = num
    return max_val
```

4. $O(n \log n)$ - Linearithmic Time

- Efficient sorting algorithms (merge sort, quick sort)
- Common in data science operations

```
# O(n log n) - Sorting
sorted_list = sorted([3, 1, 4, 1, 5, 9, 2, 6])
```

5. $O(n^2)$ - Quadratic Time

- Nested loops
- Comparing all pairs

```
# O(n^2) - Quadratic time
def find_duplicates(lst):
    duplicates = []
    for i in range(len(lst)):
        for j in range(i + 1, len(lst)):
            if lst[i] == lst[j]:
```

```
        duplicates.append(lst[i])
    return duplicates
```

6. $O(2^n)$ - Exponential Time

- Recursive algorithms without memoization
- Very slow, avoid if possible

```
# O(2^n) - Exponential (inefficient Fibonacci)
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2) # Very slow!
```

Space Complexity

Space complexity describes how much memory an algorithm uses.

```
# O(1) space - Constant space
def find_max(lst):
    max_val = lst[0] # Only one variable
    for num in lst:
        if num > max_val:
            max_val = num
    return max_val

# O(n) space - Linear space
def double_list(lst):
    return [x * 2 for x in lst] # Creates new list of size n
```

Practical Examples

Example 1: List Operations

```
# O(1) - Constant time
my_list = [1, 2, 3, 4, 5]
first = my_list[0] # Direct access

# O(n) - Linear time
total = sum(my_list) # Must visit each element

# O(n) - Linear time
my_list.append(6) # Usually O(1), but can be O(n) if list needs to resize
```

```
# O(n) - Linear time
my_list.insert(0, 0) # Must shift all elements
```

Example 2: Dictionary vs List Lookup

```
# List lookup: O(n) - must search through list
my_list = [1, 2, 3, 4, 5]
if 3 in my_list: # O(n) - checks each element
    print("Found")

# Dictionary lookup: O(1) - direct hash lookup
my_dict = {1: 'a', 2: 'b', 3: 'c', 4: 'd', 5: 'e'}
if 3 in my_dict: # O(1) - direct access
    print("Found")
```

Example 3: Optimizing Code

```
# Inefficient: O(n²)
def has_duplicate_inefficient(lst):
    for i in range(len(lst)):
        for j in range(i + 1, len(lst)):
            if lst[i] == lst[j]:
                return True
    return False

# Efficient: O(n)
def has_duplicate_efficient(lst):
    seen = set() # O(1) lookup
    for item in lst:
        if item in seen: # O(1) check
            return True
        seen.add(item) # O(1) add
    return False
```

Time Complexity in Data Science

Understanding time complexity is crucial when working with large datasets:

```
import pandas as pd
import numpy as np

# O(n) - Efficient
df['new_col'] = df['col1'] + df['col2'] # Vectorized operation

# O(n²) - Inefficient (avoid!)
```

```
for idx, row in df.iterrows(): # Very slow for large DataFrames
    df.loc[idx, 'new_col'] = row['col1'] + row['col2']
```

Key Takeaways for Time Complexity

1. **Choose the right data structure:** Lists vs Sets vs Dictionaries
 2. **Avoid nested loops:** Often leads to $O(n^2)$ or worse
 3. **Use vectorized operations:** NumPy/Pandas are optimized
 4. **Profile your code:** Use `timeit` or `cProfile` to measure actual performance
 5. **Understand trade-offs:** Sometimes $O(n \log n)$ is acceptable for simplicity
-

Iterators and Generators

Iterators and generators are powerful Python features that enable memory-efficient processing of large datasets, which is crucial in data science.

What are Iterables?

An iterable is any object that can be looped over (lists, tuples, strings, dictionaries).

```
# All of these are iterables
my_list = [1, 2, 3]
my_string = "hello"
my_dict = {'a': 1, 'b': 2}

# You can iterate over them
for item in my_list:
    print(item)
```

What are Iterators?

An iterator is an object that implements the iterator protocol:

- `__iter__()`: Returns the iterator object
- `__next__()`: Returns the next value

```
# Lists are iterables, not iterators
my_list = [1, 2, 3]
iterator = iter(my_list) # Convert to iterator

print(next(iterator)) # 1
print(next(iterator)) # 2
print(next(iterator)) # 3
# print(next(iterator)) # Raises StopIteration
```

Creating Custom Iterators

```
class Countdown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1

# Use the iterator
counter = Countdown(5)
for num in counter:
    print(num) # 5, 4, 3, 2, 1
```

What are Generators?

Generators are a simple way to create iterators using functions. They use **yield** instead of **return**.

Key Benefits:

- Memory efficient (lazy evaluation)
- Can represent infinite sequences
- Cleaner code than custom iterators

Generator Functions


```

# Regular function (eager evaluation)
def squares_list(n):
    result = []
    for i in range(n):
        result.append(i ** 2)
    return result  # Returns entire list in memory

# Generator function (lazy evaluation)
def squares_generator(n):
    for i in range(n):
        yield i ** 2  # Yields one value at a time

# Compare memory usage
list_squares = squares_list(1000)  # Stores all 1000 values
gen_squares = squares_generator(1000)  # Doesn't store anything yet!

# Generator only computes when you iterate
for square in gen_squares:
    print(square)  # Computes on-the-fly

```

Generator Expressions

Generator expressions are like list comprehensions but for generators:

```

# List comprehension (eager)
squares_list = [x**2 for x in range(10)]  # Creates list immediately

# Generator expression (lazy)
squares_gen = (x**2 for x in range(10))  # Creates generator

# Memory efficient
print(sum(squares_gen))  # Generator computes values as needed

```

Practical Examples

Example 1: Reading Large Files

```

# Inefficient: Loads entire file into memory
def read_file_inefficient(filename):
    with open(filename, 'r') as f:
        return f.readlines()  # Loads all lines at once

# Efficient: Generator reads one line at a time
def read_file_efficient(filename):
    with open(filename, 'r') as f:
        for line in f:

```

```
yield line.strip() # Yields one line at a time
```

```
# Process large file without loading into memory
for line in read_file_efficient('large_file.txt'):
    process(line) # Process one line at a time
```

Example 2: Infinite Sequences

```
# Generator for infinite Fibonacci sequence
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

# Use generator (won't run forever unless you break)
fib = fibonacci()
for i, num in enumerate(fib):
    if i >= 10: # Only get first 10
        break
    print(num) # 0, 1, 1, 2, 3, 5, 8, 13, 21, 34
```

Example 3: Processing Large Datasets

```
# Generator for processing data in chunks
def process_in_chunks(data, chunk_size=1000):
    for i in range(0, len(data), chunk_size):
        yield data[i:i + chunk_size]

# Process large dataset in chunks
large_dataset = list(range(1000000))
for chunk in process_in_chunks(large_dataset, chunk_size=10000):
    process_chunk(chunk) # Process 10k items at a time
```

Example 4: Pipeline Processing

```
# Generator pipeline for data processing
def read_numbers(filename):
    with open(filename, 'r') as f:
        for line in f:
            yield int(line.strip())

def filter_even(numbers):
    for num in numbers:
        if num % 2 == 0:
            yield num

def square(numbers):
    for num in numbers:
```

```
yield num** 2
```

```
# Chain generators (memory efficient)
numbers = read_numbers('numbers.txt')
evens = filter_even(numbers)
squares = square(evens)
```

```
# Process on-the-fly
for square_val in squares:
    print(square_val)
```

Generator vs Iterator

```
# Generator function (simpler)
def count_up_to(max):
    count = 1
    while count <= max:
        yield count
        count += 1

# Custom iterator (more verbose)
class CountUpTo:
    def __init__(self, max):
        self.max = max
        self.count = 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.count > self.max:
            raise StopIteration
        current = self.count
        self.count += 1
        return current

# Both work the same way
for num in count_up_to(5):
    print(num) # 1, 2, 3, 4, 5
```

When to Use Generators

Use generators when:

- Working with large datasets
- Processing data in streams
- Creating infinite sequences

- Memory is a concern
- You want lazy evaluation

Use lists when:

- You need random access (indexing)
- You need to iterate multiple times
- Dataset is small
- You need list methods (append, extend, etc.)

Generator Best Practices

```
# Good: Generator for large data
def process_large_file(filename):
    with open(filename, 'r') as f:
        for line in f:
            yield process_line(line)

# Good: Generator expression for filtering
large_list = range(1000000)
evens = (x for x in large_list if x % 2 == 0)

# Avoid: Converting generator to list unnecessarily
# bad: list(gen) if you only need to iterate once
# good: iterate directly over generator
```

Key Takeaways for Iterators and Generators

1. **Generators are memory efficient:** Process data one item at a time
2. **Use `yield` for generators:** Creates iterator automatically
3. **Generator expressions are concise:** `(x**2 for x in range(10))`
4. **Perfect for large datasets:** Don't load everything into memory
5. **Can represent infinite sequences:** Useful for streaming data
6. **Chain generators:** Create processing pipelines

GUI Development with tkinter

Introduction to tkinter

tkinter is Python's built-in GUI (Graphical User Interface) library. It allows you to create desktop applications with windows, buttons, and other widgets.

Why Learn GUI Development?

- **Desktop Applications:** Create standalone applications
- **User Interfaces:** Build interactive tools
- **Data Visualization:** Create custom visualization tools
- **Prototyping:** Quick UI prototypes

Basic tkinter Application

```
import tkinter as tk
from tkinter import ttk

# Create main window
root = tk.Tk()
root.title("My First GUI")
root.geometry("400x300")

# Add label
label = tk.Label(root, text="Hello, tkinter!", font=("Arial", 16))
label.pack(pady=20)

# Add button
def button_clicked():
    label.config(text="Button clicked!")

button = tk.Button(root, text="Click Me", command=button_clicked)
button.pack(pady=10)

# Run application
root.mainloop()
```

Common Widgets

Entry (Text Input):

```
entry = tk.Entry(root, width=30)
entry.pack()

def get_text():
    text = entry.get()
    print(f"Entered: {text}")
```

```
button = tk.Button(root, text="Get Text", command=get_text)
button.pack()
```

Text Widget (Multi-line):

```
text_widget = tk.Text(root, width=40, height=10)
text_widget.pack()

# Get all text
content = text_widget.get("1.0", tk.END)
```

Checkbox:

```
var = tk.BooleanVar()
checkbox = tk.Checkbutton(root, text="I agree", variable=var)
checkbox.pack()

def check_value():
    print(f"Checked: {var.get()}")
```

Radio Buttons:

```
var = tk.StringVar(value="option1")

radio1 = tk.Radiobutton(root, text="Option 1", variable=var, value="option1")
radio1.pack()

radio2 = tk.Radiobutton(root, text="Option 2", variable=var, value="option2")
radio2.pack()
```

Listbox:

```
listbox = tk.Listbox(root)
listbox.pack()

# Add items
for item in ["Item 1", "Item 2", "Item 3"]:
    listbox.insert(tk.END, item)

# Get selected
def get_selected():
    selection = listbox.curselection()
    if selection:
        print(listbox.get(selection[0]))
```

Layout Managers

Pack (Simple):

```
label1 = tk.Label(root, text="Label 1")
label1.pack(side=tk.LEFT)

label2 = tk.Label(root, text="Label 2")
label2.pack(side=tk.RIGHT)
```

Grid (Table-like):

```
label = tk.Label(root, text="Row 0, Col 0")
label.grid(row=0, column=0)

button = tk.Button(root, text="Row 1, Col 0")
button.grid(row=1, column=0)
```

Place (Absolute):

```
label = tk.Label(root, text="At (100, 50)")
label.place(x=100, y=50)
```

Example: Simple Calculator

```
import tkinter as tk

def calculate():
    try:
        num1 = float(entry1.get())
        num2 = float(entry2.get())
        result = num1 + num2
        result_label.config(text=f"Result: {result}")
    except ValueError:
        result_label.config(text="Invalid input")

root = tk.Tk()
root.title("Simple Calculator")

entry1 = tk.Entry(root, width=10)
entry1.pack(pady=5)

entry2 = tk.Entry(root, width=10)
entry2.pack(pady=5)
```

```
button = tk.Button(root, text="Add", command=calculate)
button.pack(pady=5)

result_label = tk.Label(root, text="Result: ")
result_label.pack(pady=5)

root.mainloop()
```

Data Science Application Example

```
import tkinter as tk
from tkinter import filedialog
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

def load_data():
    filename = filedialog.askopenfilename(
        filetypes=[("CSV files", "*.csv")]
    )
    if filename:
        df = pd.read_csv(filename)
        # Display data
        print(df.head())
        return df
    return None

def plot_data():
    df = load_data()
    if df is not None:
        fig, ax = plt.subplots(figsize=(6, 4))
        df.plot(ax=ax)

        # Embed in tkinter
        canvas = FigureCanvasTkAgg(fig, root)
        canvas.draw()
        canvas.get_tk_widget().pack()

root = tk.Tk()
root.title("Data Science Tool")

load_button = tk.Button(root, text="Load CSV", command=load_data)
load_button.pack(pady=10)

plot_button = tk.Button(root, text="Plot Data", command=plot_data)
plot_button.pack(pady=10)

root.mainloop()
```


Key Takeaways

1. **tkinter is built-in**: No installation needed
 2. **Widgets**: Labels, buttons, entries, etc.
 3. **Layout**: Pack, Grid, or Place
 4. **Events**: Use commands for button clicks
 5. **Simple GUIs**: Great for quick tools
-

Key Takeaways

1. **Python is dynamically typed** - no need to declare variable types
 2. **Indentation matters** - Python uses indentation for code blocks
 3. **Lists are versatile** - most commonly used data structure
 4. **Dictionaries are powerful** - key-value pairs for structured data
 5. **Functions are first-class** - can be passed as arguments
 6. **Error handling is important** - use try/except for robust code
 7. **OOP helps organize code** - classes and objects for complex programs
-

Next Steps

- Practice writing Python programs daily
- Work through the exercises above
- Try solving problems on [HackerRank](#) or [LeetCode](#)
- Move to [02-linear-algebra.md](#) when comfortable

Remember: Practice is key! Code along with examples and experiment with variations.